EECS 4422 Computer Vision

Frequency-Domain Deblurring

Simon Posluns - 221541727 - simonp22@my.yorku.ca

Implementing and comparing Fourier-domain restoration methods for deblurring

Image source

Clean reference images are taken from **CBSD68** (68 colour natural images, 321×481), obtained from the benchmark collection at `https://github.com/majedelhelou/denoising_datasets`.

Degradation model | Synthetically blurring the input images

Following the course notation, g denotes the original image and f the degraded observation:

$$f(x,y) = h(x,y) * g(x,y) + n(x,y) \tag{1}$$
$$F(u,v) = H(u,v)\, G(u,v) + N(u,v) \tag{2}$$

g = original (sharp) image, h = blur kernel (PSF), n = additive noise, f = observed blurry image; capitals denote the corresponding 2-D discrete Fourier transforms, and $\hat{G}$ denotes the restored estimate of G.

What the blurring operation does

Applies a convolution:

$$(h * g)(x,y) = \sum_i \sum_j h(i,j)\, g(x-i,\ y-j) \tag{3}$$

The kernel used here is an isotropic Gaussian of width σ, normalised to unit sum:

$$h(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right), \qquad \sum_{x,y} h(x,y) = 1 \tag{4}$$

Because the weights sum to one, the local average is preserved, so blurring redistributes intensity without changing overall brightness. The kernel is evaluated on an odd window of half-width $\lceil 3\sigma \rceil$, beyond which its values are negligible.

The filter is applied separately to each of the three colour channels, and the convolution is *circular*: the image is treated as periodic, so the same H that produced the blur can invert it exactly. Zero- or reflection-padded convolution would introduce boundary behaviour that no frequency-domain filter could undo.

What the noise term is

$n(x,y)$ is additive white Gaussian noise, drawn independently per pixel and per channel after the blur has been applied:

$$n(x,y) \sim \mathcal{N}\big(0,\ \sigma_n^2\big), \qquad \mathbb{E}[n] = 0 \tag{5}$$

Being white, its power is spread evenly over the spectrum, so P_n is flat — the fact both restoration filters below rely on. Noise strength is quoted on the 0–255 scale used throughout the denoising literature, where $\sigma_n = 15$, 25 and 50 are the standard CBSD68 operating points; images are held as floats in $[0,1]$, so the value used in computation is $\sigma_n/255$. Results here use $\sigma_n = 15$ unless stated.

Degradation is carried out in double precision and the resulting array is passed to the restoration filters directly, with no intermediate 8-bit write. The perturbation the filters face is therefore exactly the n the model above specifies, and nothing else; the PNGs in `blurred/` exist only to be displayed in this report. The blurred image is clipped to $[0,1]$ before n is added — a non-negative unit-sum kernel cannot leave that range, so this removes float round-off only — and the sum is left unclipped, so n remains exactly Gaussian.

Inverse filter

By the convolution theorem, convolution in the spatial domain becomes multiplication in the frequency domain:

$$g * h \quad \Longleftrightarrow \quad G\,H \tag{6}$$

Applying this to the degradation model turns the convolution into a product:

$$f = h * g + n \quad \Longleftrightarrow \quad F = H\,G + N \tag{7}$$

Considering the Fourier transforms of g, f and h, and assuming for the moment that $N = 0$:

$$G(u,v) = \text{the sharp image to recover,} \tag{8}$$
$$F(u,v) = \text{the blurred image} \tag{9}$$
$$H(u,v) = \text{the blur, as a gain applied to each frequency.} \tag{10}$$

Blurring multiplies G by H, so the restoration divides it back out:

$$\hat{G} = \frac{F}{H} \tag{11}$$

Since $|H|$ becomes vanishingly small at high frequencies, it is floored at ε before dividing (to avoid division by 0):

$$\hat{G}(u,v) = \frac{F(u,v)}{H_\varepsilon(u,v)}, \qquad H_\varepsilon = \begin{cases} H, & |H| \geq \varepsilon, \\ \varepsilon, & |H| < \varepsilon, \end{cases} \qquad \varepsilon = 10^{-2} \tag{12}$$

The assumption $N = 0$ is false by construction here, and the consequence is severe. Dividing $F = HG + N$ by H returns $G + N/H$: the noise is divided by the same vanishing $|H|$, so wherever the blur has destroyed signal the filter amplifies noise instead of recovering detail. The ε-floor bounds that amplification by $1/\varepsilon$ but cannot prevent it, and no choice of ε escapes the trade-off — lowering it amplifies more noise, raising it leaves more blur. This filter is included as the baseline that motivates the two that follow, not as a working method.

Steps

1. Build the kernel h from σ, then transform it: $H = \mathcal{F}\{h\}$. A Gaussian is fully determined by σ, so no other information is needed, and this is the identical H that produced the blur.
2. Floor it: $H_\varepsilon = \max(|H|,\ \varepsilon)$, keeping H elsewhere.
3. For each colour channel:
   (a) $F = \mathcal{F}\{f\}$ (pixels $\rightarrow$ frequencies)
   (b) $\hat{G} = F/H_\varepsilon$ (divide out the blur)
   (c) $\hat{g} = \operatorname{Re} \mathcal{F}^{-1}\{\hat{G}\}$ (frequencies $\rightarrow$ pixels)
   (d) keep the real part of output, discard the imaginary part,

Wiener filter

Rather than assume $n = 0$, find the linear shift-invariant filter w minimising the expected mean square error between the estimate $\hat{g} = w * f$ and the original image g:

$$\min_{w} \ \mathbb{E}\left[\iint \big(\hat{g}(x,y) - g(x,y)\big)^2 dx\, dy \right] \tag{13}$$

The solution is expressed in terms of *power spectral densities* (PSDs) — the expected energy each signal carries at each frequency — of the original image, the noise, and the degraded image respectively:

$$P_g(u,v) = \mathbb{E}\Big[|G(u,v)|^2\Big] \tag{14}$$
$$P_n(u,v) = \mathbb{E}\Big[|N(u,v)|^2\Big] \tag{15}$$
$$P_f(u,v) = \mathbb{E}\Big[|F(u,v)|^2\Big] \tag{16}$$

Substituting $F = HG + N$ into $P_f = \mathbb{E}\big[|F|^2\big]$ and expanding with $|a+b|^2 = |a|^2 + |b|^2 + 2\operatorname{Re}(ab^*)$:

$$\begin{aligned} P_f &= \mathbb{E}\Big[|HG + N|^2\Big] \\ &= |H|^2\, \mathbb{E}\Big[|G|^2\Big] + \mathbb{E}\Big[|N|^2\Big] + 2\operatorname{Re}\mathbb{E}[HGN^*] \end{aligned} \tag{17}$$

The cross term vanishes because H is deterministic, G and N are independent, and the noise has zero mean:

$$\mathbb{E}[HGN^*] = H\,\mathbb{E}[GN^*] = H\,\mathbb{E}[G]\,\mathbb{E}[N^*] = 0 \tag{18}$$

Dropping it leaves the two remaining terms:

$$P_f = |H|^2\, \mathbb{E}\Big[|G|^2\Big] + \mathbb{E}\Big[|N|^2\Big] \tag{19}$$

which are P_g and P_n as defined above. Thus:

$$P_f = |H|^2\, P_g + P_n \tag{20}$$

The best W is the one that leaves an error independent of the data: if the error still depended on the data, adjusting W could remove more of it.

$$\mathbb{E}[(WF - G)\,F^*] = 0 \tag{21}$$

Expanding the product and splitting the expectation, W being a deterministic scalar at each frequency:

$$\begin{aligned} \mathbb{E}\Big[W\,|F|^2\Big] - \mathbb{E}[GF^*] &= 0 \\ W\,\mathbb{E}\Big[|F|^2\Big] &= \mathbb{E}[GF^*] \end{aligned} \tag{22}$$

so that

$$W = \frac{\mathbb{E}[GF^*]}{\mathbb{E}\Big[|F|^2\Big]} \tag{23}$$

The denominator is P_f, already found above. For the numerator, substitute $F^* = (HG + N)^* = H^*G^* + N^*$:

$$\begin{aligned}\mathbb{E}[GF^*] &= \mathbb{E}[G\,(H^*G^* + N^*)] \\ &= H^*\mathbb{E}\Big[|G|^2\Big] + \mathbb{E}[GN^*] \\ &= H^*P_g \end{aligned} \tag{24}$$

the second term vanishing by the same argument as before: G and N are independent and $\mathbb{E}[N^*] = 0$. Substituting both:

$$W = \frac{\mathbb{E}[GF^*]}{\mathbb{E}\Big[|F|^2\Big]} = \frac{H^*P_g}{P_f} = \frac{H^*P_g}{|H|^2\,P_g + P_n} \tag{25}$$

That is, $W \approx 1/H$ where the signal dominates and $W \approx 0$ where the noise does.
P_g appears in both the numerator and the denominator, so dividing through by it removes it from the numerator entirely and leaves the two PSDs appearing only as their ratio:

$$W = \frac{H^*P_g}{|H|^2\,P_g + P_n} = \frac{H^*}{|H|^2 + \dfrac{P_n}{P_g}} \tag{26}$$

This is the step that makes the filter usable: the individual spectra are unknown, but only their ratio is now required.

Considering P_n/P_g: *Why do you sub σ_n^2 for P_n and $\mathrm{Var}(f)$ for P_g?*

Because the two are the same quantity viewed from either domain. By Parseval's theorem the total energy of a zero-mean signal is the same computed over pixels or over frequencies:

$$\sum_{x,y} |s(x,y)|^2 = \frac{1}{MN}\sum_{u,v} |S(u,v)|^2 \tag{27}$$

For the noise the substitution is exact. White noise carries equal power at every frequency, so its per-frequency power is simply its variance — a single number is not an approximation, it is the entire spectrum:

$$P_n(u,v) = \sigma_n^2 \qquad \text{for all } (u,v) \tag{28}$$

For the image it is an approximation. $\mathrm{Var}(f)$ is the total power of the observation spread across all frequencies, whereas P_g is strongly frequency-dependent: natural images concentrate most of their energy at low frequencies and carry very little at high ones. Replacing that curve with its average is exactly what treating K as constant assumes, and it is the reason the practical filter under-damps low frequencies and over-damps high ones relative to the oracle described below.

Writing $K = P_n/P_g$ for that ratio:

$$W(u,v) = \frac{H^*(u,v)}{|H(u,v)|^2 + K} \tag{29}$$

and applying it, $\hat{G} = WF$:

$$\hat{G}(u,v) = \frac{H^*(u,v)}{|H(u,v)|^2 + K}\,F(u,v) \tag{30}$$

The denominator cannot reach zero, so no ε-floor is needed;

Steps

1. Build the kernel h from σ, then transform it: $H = \mathcal{F}\{h\}$.
2. Form the filter once: $W = H^*/\Big(|H|^2 + K\Big)$.
3. For each colour channel:
   (a) $F = \mathcal{F}\{f\}$ (pixels $\to$ frequencies)
   (b) $\hat{G} = W\,F$ (apply the filter)
   (c) $\hat{g} = \mathrm{Re}\;\mathcal{F}^{-1}\{\hat{G}\}$ (frequencies $\to$ pixels)

Note: both the estimate above and the true per-frequency PSDs are run, as `wiener_filter` and `wiener_filter_oracle`. The oracle keeps the ratio P_n/P_g rather than collapsing it, taking $P_g = |G|^2$ from the ground-truth image and $P_n = MN\sigma_n^2$, flat because the noise is white. Since P_g requires the image being recovered, the oracle is an upper bound on what a Wiener filter could achieve, not an achievable result; the gap between the two rows is the cost of not knowing the signal spectrum.

Constrained least squares (Laplacian)

The Wiener filter needs P_n and P_g, which are unknown in general. Constrained least squares replaces those statistics with a preference: among all estimates consistent with the data, choose the smoothest. Roughness is measured by the Laplacian, ∇^2, the sum of the second derivative along each axis:

$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \tag{31}$$

Differencing twice along x gives the discrete form of each term:

$$\begin{aligned}\frac{\partial^2 g}{\partial x^2} &\approx \big[g(x{+}1) - g(x)\big] - \big[g(x) - g(x{-}1)\big] \\ &= g(x{+}1) - 2g(x) + g(x{-}1)\end{aligned} \tag{32}$$

so each axis contributes the kernel $[\,1\ \ {-2}\ \ 1\,]$, and their sum is the 3×3 operator p:

$$p = \begin{bmatrix} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \tag{33}$$

Its weights sum to zero, so a region of constant intensity gives exactly zero response: the operator measures curvature only, and is blind to brightness.

Minimise that roughness subject to the estimate reproducing the observed image:

$$\min_{\hat g}\ \|p * \hat g\|^2 \qquad \text{subject to} \qquad \|f - h * \hat g\|^2 = \|n\|^2 \tag{34}$$

Introducing a multiplier γ for the constraint and setting the derivative of the combined objective to zero gives, in the frequency domain:

$$\hat G(u,v) = \frac{H^*(u,v)}{|H(u,v)|^2 + \gamma\,|P(u,v)|^2}\ F(u,v) \tag{35}$$

where $P = \mathcal{F}\{p\}$. This is the Wiener filter with the constant K replaced by $\gamma\,|P|^2$. Because p is high-pass, that penalty varies with frequency:

$$P(0,0) = 0 \qquad \text{the mean is unconstrained,} \tag{36}$$

$$|P|^2 \to 64 \qquad \text{maximal penalty at the highest frequencies.} \tag{37}$$

So the damping is applied exactly where the inverse filter fails, and no *spectral* noise statistics are required — only the assumption that the original image is smooth, plus a single scalar setting how hard to press it.

That scalar still has to come from somewhere. The constraint above fixes it in principle: γ should be the value at which the residual $\|f - h * \hat g\|^2$ equals the known noise power, found by iteration. Here it is instead set directly, as a fixed multiple of the same measurable ratio the Wiener filter uses:

$$\gamma = c\,\frac{\sigma_n^2}{\mathrm{Var}(f)}, \qquad c = 8 \tag{38}$$

The multiplier is needed because $|P|^2$ reaches 64 rather than 1, so γ and K are not on the same scale; $c = 8$ was chosen by sweeping γ against PSNR at $\sigma_n = 5, 15, 25, 50$ and taking a value near the optimum at every level. PSNR is flat enough around the peak that the loss against a per-level optimum is under 0.2 dB, and this avoids re-tuning by hand whenever σ_n changes.

Steps

1. Build the kernel h from σ and transform it: $H = \mathcal{F}\{h\}$.
2. Build the Laplacian p on the same grid and transform it: $P = \mathcal{F}\{p\}$.
3. Form the filter once: $W = H^* / \left(|H|^2 + \gamma\,|P|^2\right)$.
4. For each colour channel:
   (a) $F = \mathcal{F}\{f\}$ (pixels $\to$ frequencies)
   (b) $\hat G = W\,F$ (apply the filter)
   (c) $\hat g = \mathrm{Re}\ \mathcal{F}^{-1}\{\hat G\}$ (frequencies $\to$ pixels)

Results

All four filters are run over the same 68 CBSD68 images, degraded with $\sigma = 2$ blur and $\sigma_n = 15$ additive white Gaussian noise. Every filter sees the identical noise realisation, drawn from a generator seeded by file name, so the comparison is not confounded by the draw. Each restored image is clipped to $[0, 1]$ before scoring. PSNR measures squared error; SSIM compares local mean, contrast and correlation over a 7×7 window, so the two disagree where a method trades error for structure. Runtimes are per image, averaged over the set, and exclude file I/O.

Method	PSNR (dB)	Δ PSNR	SSIM	ms/image
Degraded input	21.24	—	0.3289	—
Inverse filter	5.30	-15.95	0.0061	22
Wiener filter	22.12	+0.88	0.5480	22
Wiener filter (oracle)	24.60	+3.36	0.6266	34
Constrained least squares	24.08	+2.83	0.6078	26

Table 1. Mean PSNR and SSIM over 68 images, $\sigma = 2$, $\sigma_n = 15$.

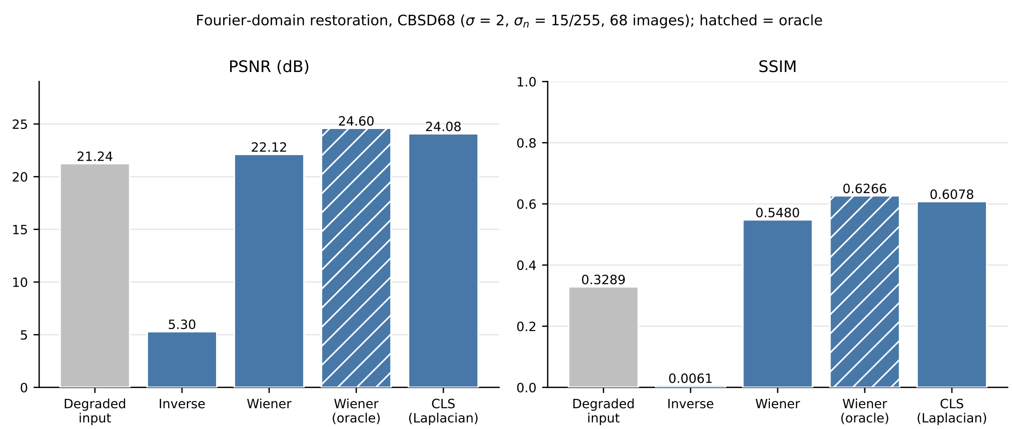


Figure 1. The same figures. The oracle (hatched) uses the ground-truth P_g, so it is an upper bound rather than an achievable result.

Three things stand out. The inverse filter lands roughly $16\,\mathrm{dB}$ *below* its own input: with $N \neq 0$ the division amplifies noise faster than it removes blur, and the ε-floor only bounds the damage. The two damped filters both beat the degraded input, and constrained least squares comes within $0.5\,\mathrm{dB}$ of the oracle while using no knowledge of the ground truth — the smoothness prior turns out to be a close stand-in for the true P_g on natural images. Runtimes are within a few milliseconds of each other, since all four are dominated by the same two FFTs per channel; the oracle is slower only because it transforms the ground-truth image as well.

Sample restorations

One grid per image, each cell captioned with its own PSNR and SSIM against the ground truth. Generated by `save_panels()`, so the scores below cannot drift from those in Table 1.

Degraded input — 24.03 dB / 0.2475

Inverse filter — 6.05 dB / 0.0041

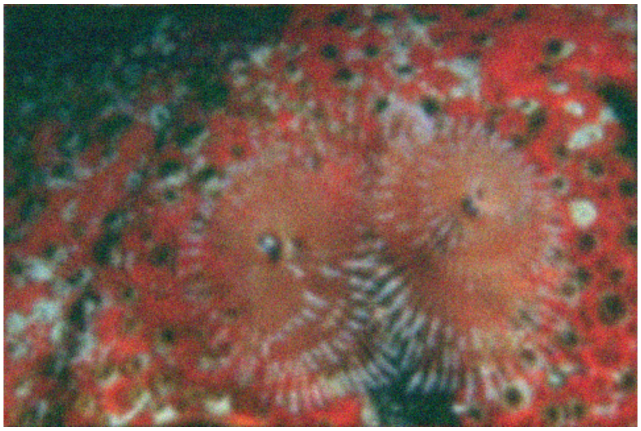
Wiener filter — 19.89 dB / 0.7999

Wiener filter (oracle) — 32.01 dB / 0.8789

Constrained least squares — 31.54 dB / 0.8654

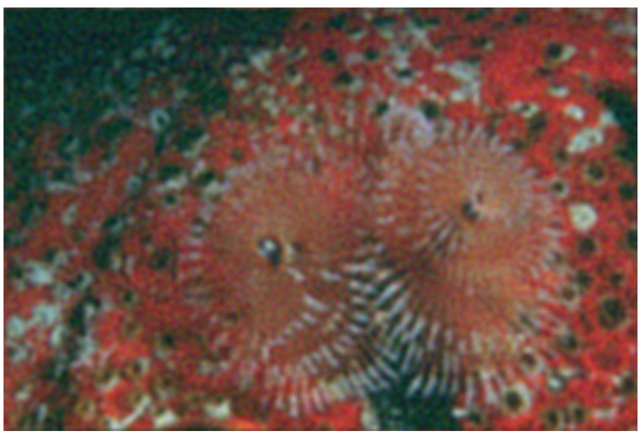
Ground truth

Figure 2. `3096.png` restored from $\sigma = 2$ blur with $\sigma_n = 15$ noise. Read left to right, top to bottom.

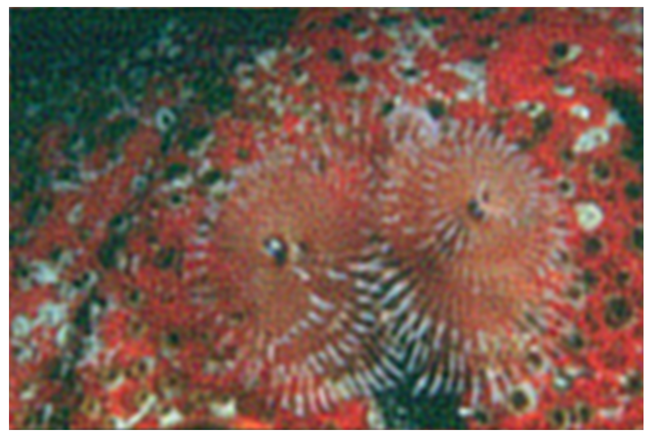


Degraded input — 21.48 dB / 0.3740

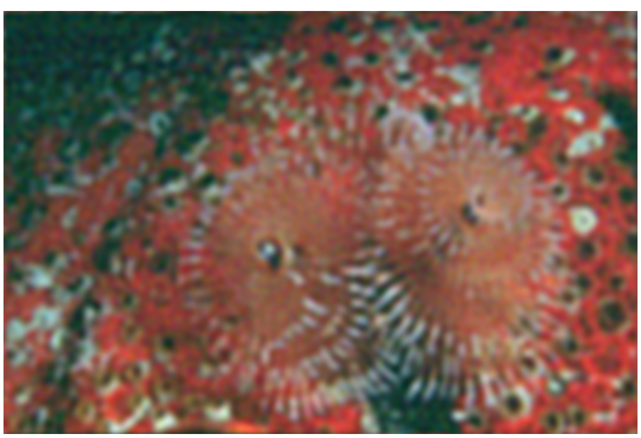
Inverse filter — 5.54 dB / 0.0066



Wiener filter — 22.38 dB / 0.5984



Wiener filter (oracle) — 24.79 dB / 0.6370



Constrained least squares — 24.08 dB / 0.6015

Ground truth

Figure 3. `12084.png` restored from $\sigma = 2$ blur with $\sigma_n = 15$ noise. Read left to right, top to bottom.

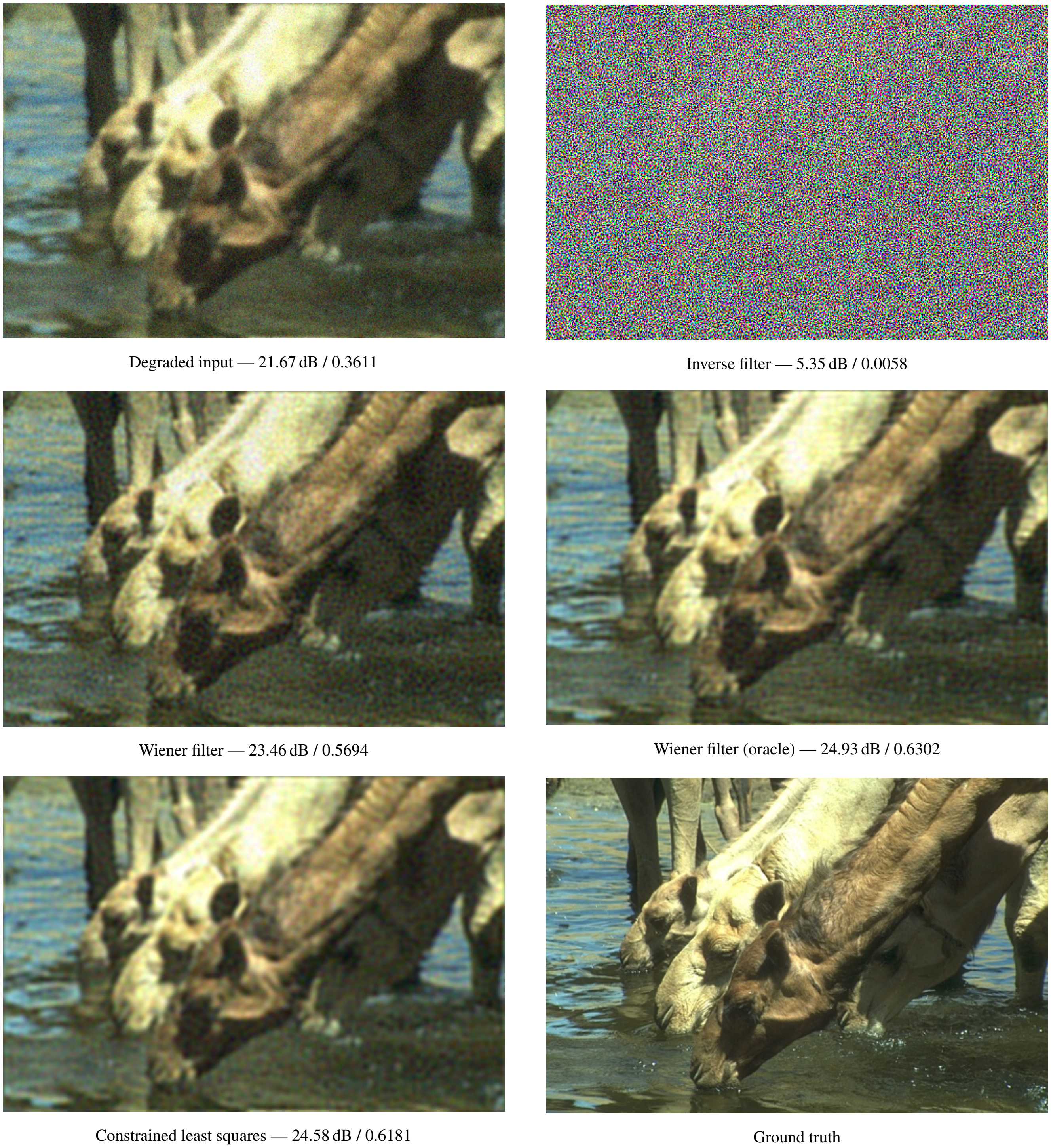


Figure 4. `16077.png` restored from $\sigma = 2$ blur with $\sigma_n = 15$ noise. Read left to right, top to bottom.

Degraded input — 21.27 dB / 0.3411

Inverse filter — 5.29 dB / 0.0067

Wiener filter — 22.66 dB / 0.5749

Wiener filter (oracle) — 24.24 dB / 0.6055

Constrained least squares — 23.79 dB / 0.6062

Ground truth

Figure 5. `19021.png` restored from $\sigma = 2$ blur with $\sigma_n = 15$ noise. Read left to right, top to bottom.

Degraded input — 19.97 dB / 0.4275

Inverse filter — 4.94 dB / 0.0091

Wiener filter — 22.15 dB / 0.6063

Wiener filter (oracle) — 23.34 dB / 0.6771

Constrained least squares — 22.45 dB / 0.6980

Ground truth

Figure 6. `24077.png` restored from $\sigma = 2$ blur with $\sigma_n = 15$ noise. Read left to right, top to bottom.